



Funded by the
European Union

HuNavSim 2.0

Deliverable 9 - HuNavSim documentation

December 2025

Revision History

Revision	Date	Author(s)	Description
1.0	09.12.2025	Noé Pérez-Higueras	Content sections created
1.1	12.12.2025	Miguel Escudero	First version
2.0	19.12.2025	Noé Pérez-Higueras	Reviewed and edited

Contents

1	Introduction	4
2	HuNavSim architecture	5
2.1	Simulator wrapper interface	5
2.2	HuNav Manager: behavior execution and social dynamics	6
2.3	HuNav Loader: scenario initialization and parameter management	7
2.4	HuNav Evaluator and configuration tools	8
3	HuNavSim distribution and installation	8
3.1	Hardware and software requirements	8
3.1.1	Software Dependencies	9
3.1.2	Hardware Requirements	9
3.2	Docker containers	10
3.3	Installation steps	10
3.3.1	Prerequisites	11
3.3.2	Automated Installation Process	11
4	Execution of predefined scenarios	12
4.1	Available Scenarios	12
4.2	Launching Scenarios	14
4.2.1	Docker-Based Execution	14
4.2.2	Native ROS 2 Execution	15
5	Creation of new scenarios	17
5.1	RViz panel for scenario generation	17
5.2	Assisted generation of human behaviors	21
5.2.1	Visual behavior composition wizard	21
5.2.2	LLM-based behavior generation	25
5.2.3	Comparative workflow and integration	25
5.3	Assisted Gazebo environment generation	26
5.3.1	Using natural language	26
5.3.2	Using a ROS 2 map	28
5.4	Creation of ROS 2 navigation maps	30
6	Benchmarking	32
6.1	RViz panel for metrics configuration	32
6.2	Metric computation during simulation	32
6.3	Results generation and analysis	34
6.4	Adding new metrics	36

1 Introduction

Social navigation—the ability of mobile robots to navigate safely, efficiently, and legibly in human-populated environments—remains a fundamental challenge in human-robot interaction. Effective development and evaluation of social navigation algorithms requires simulation frameworks capable of generating realistic, diverse, and controllable human behaviors while providing quantitative benchmarking capabilities. *HuNavSim* (Human Navigation Simulator) addresses this need through an open-source, simulator-agnostic framework for social robot navigation research under the ROS 2 ecosystem.

This work presents *HuNavSim 2.0*, a major architectural and functional advancement over the previous version. The original framework employed the Social Force Model (SFM) [1] for continuous motion generation combined with a limited set of predefined behavior trees encoding simple reactive behaviors (for instance, standard goal-directed navigation, exploratory or “curious” motion patterns, and avoidance- or “scared”-like navigation behaviors, among others). While this approach provided computationally efficient collision avoidance and basic behavioral differentiation, the constrained behavior tree node library and fixed behavioral templates restricted agents to predetermined response patterns, limiting behavioral diversity and precluding semantic, contextual, and personality-driven adaptations to dynamic social situations.

HuNavSim 2.0 extends this foundation through three principal enhancements: (1) an expanded behavior tree node library exposing rich environmental perception, social interaction primitives, and agent state management capabilities; (2) interactive visual tools for composing custom behavior trees from modular functional blocks without manual XML editing; and (3) Large Language Model (LLM) integration enabling natural-language-driven behavior synthesis and automated generation of scenario-specific agent policies. This augmented architecture preserves the reactive stability of SFM for local collision avoidance while enabling agents to exhibit complex social interactions—goal-oriented navigation with adaptive replanning, reactive responses to environmental stimuli, personality-driven behaviors, context-aware decision-making, and emergent social dynamics—through user-defined or LLM-generated behavior tree compositions.

The framework implements a simulator-agnostic architecture through a wrapper layer that decouples social navigation logic from physics simulation and rendering. This abstraction enables deployment across four major robotics simulators—Gazebo Classic, Gazebo Fortress, NVIDIA Isaac Sim, and Webots—with unified agent behavior specifications and evaluation protocols. Interactive scenario authoring is provided through dedicated RViz 2 panels implementing a *what-you-see-is-what-you-simulate* workflow: users visually configure agent populations, assign navigation goals through map clicking, compose or generate behavior trees via wizard interfaces or LLM prompts, and export complete scenarios to YAML/XML representations. A comprehensive evaluation module computes 33 social navigation metrics spanning proxemic compliance (Hall’s theory-based space intrusions), motion quality (acceleration profiles, heading

stability), task efficiency (path length, completion time), safety (collision detection), social dynamics (force interactions), and psychological comfort (danger and surprise indices), facilitating systematic benchmarking and reproducible algorithm comparison.

HuNavSim 2.0 aims to reduce the barrier to entry for social navigation research while supporting advanced experimentation through progressive disclosure of complexity. Novice users can leverage predefined behavior templates and the visual configuration interface to rapidly prototype scenarios, while experienced researchers can extend the behavior tree node library, define custom metrics, or integrate novel planning algorithms through the modular ROS 2 architecture. The framework’s integrated toolchain—from visual scenario design through automated behavior synthesis to real-time metric computation—compresses the experimental iteration cycle, enabling researchers to progress from initial concept to quantitative validation within a single interactive session. All components are open-source and documented, fostering community contributions and supporting reproducible research practices in human-aware mobile robotics.

2 HuNavSim architecture

HuNavSim 2.0 implements a layered, simulator-agnostic architecture that decouples high-level social navigation reasoning from low-level physics simulation and rendering (Fig. 1). This separation-of-concerns design abstracts simulator-specific APIs behind a uniform ROS 2 service interface, enabling deployment across heterogeneous platforms—Gazebo Classic, Gazebo Fortress, NVIDIA Isaac Sim, and Webots—with identical agent behavior specifications, scenario definitions, and evaluation protocols. The architecture comprises four principal components operating in a coordinated pipeline: (1) *Simulator Wrappers* translate between native simulation APIs and standardized ROS 2 messages; (2) the *HuNav Manager* executes per-agent behavior trees and computes Social Force Model dynamics; (3) the *HuNav Loader* initializes scenarios from YAML configurations generated via the RViz 2 panel; and (4) the *HuNav Evaluator* subscribes to navigation topics for real-time metric computation. This modular design isolates platform-specific code within thin wrapper layers while concentrating behavioral complexity in reusable, simulator-independent components, facilitating portability, testing, and community extension.

2.1 Simulator wrapper interface

The simulator wrapper layer abstracts platform-specific APIs behind a uniform ROS 2 service interface defined in the `hunav_msgs` package. At each simulation timestep, the wrapper queries agent states from the native simulator (positions, velocities, orientations) and invokes the `/compute_agents` service, providing the complete environment state including robot odometry and obstacle information. The HuNav Manager processes this request, computes social

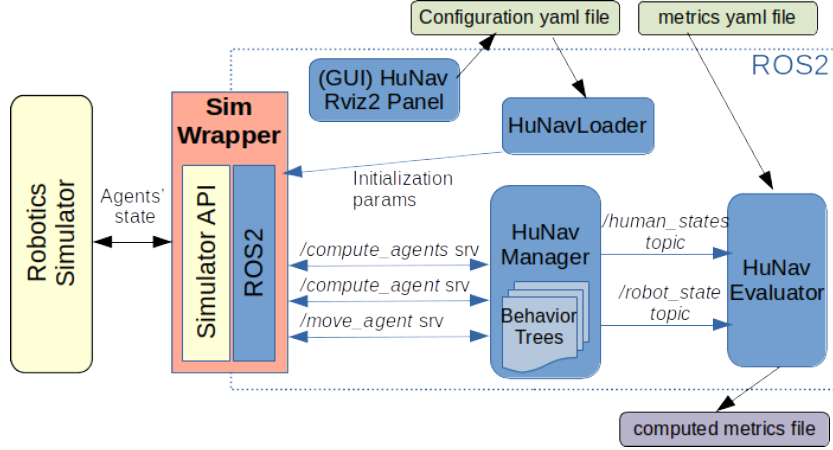


Figure 1: Architectural overview of HuNavSim 2.0. The Simulator Wrapper provides a simulator-agnostic ROS 2 interface, translating native simulation state into standardized service calls. The HuNav Manager executes per-agent behavior trees and computes Social Force Model dynamics, returning updated agent states to the simulator. The HuNav Loader initializes scenarios from YAML configurations generated via the RViz 2 panel, while the HuNav Evaluator subscribes to navigation topics for real-time metric computation. Dashed lines indicate ROS 2 boundaries; solid arrows denote primary data flows.

forces and evaluates behavior tree transitions for all agents, and returns updated agent motion commands. The wrapper then translates these commands into simulator-native control primitives. A complementary `/move_agent` service enables selective computation of individual agent states when only a subset of the agent population requires behavioral updates, reducing computational overhead in scenarios where most agents maintain static or predictable behaviors. This abstraction isolates the core social navigation logic from simulator-specific implementation details, ensuring portability across platforms with minimal code duplication and enabling seamless migration between simulation environments.

2.2 HuNav Manager: behavior execution and social dynamics

The HuNav Manager constitutes the core reasoning engine, orchestrating per-agent behavior trees and computing Social Force Model dynamics in a unified control loop. Each agent is assigned a dedicated behavior tree instance loaded from XML files (generated via the RViz 2 panel) that encodes high-level behavioral policies—goal-directed navigation, social interaction patterns (conversation formation, following, blocking), environmental responses, and personality-driven reactions (curiosity, fear, aggression). The manager maintains a behavior tree executor per agent, indexed by agent ID, evaluating condition nodes (goal

achievement, proximity detection, visibility queries, probabilistic triggers), executing action nodes (navigation with different personality profiles, waypoint selection, movement control, social approach behaviors), and managing state transitions through a blackboard mechanism that shares agent state and time deltas across the tree hierarchy.

At each control cycle, the manager integrates behavior tree outputs with Social Force Model computations to produce coherent agent motion. The SFM library calculates four force components—desired forces toward waypoints, social forces for personal space maintenance, obstacle avoidance forces, and group cohesion forces—synthesized according to agent-specific parameters (force factors, relaxation times, interaction radii, desired velocities) loaded from the scenario YAML configuration. The resulting net force vector is integrated via the SFM’s `updatePosition()` method to produce velocity and position updates, with behavior-specific velocity modulation enabling personality-driven motion (e.g., increased velocity for scared agents, reduced velocity for curious agents). Linear and angular velocities are then transmitted to the simulator wrapper. This hierarchical control architecture—behavior trees for discrete decision logic, Social Force Model for continuous motion generation—enables rich, context-aware agent behaviors while maintaining computational efficiency suitable for real-time multi-agent simulation.

2.3 HuNav Loader: scenario initialization and parameter management

The HuNav Loader node manages scenario configurations through a ROS 2 parameter server and service architecture. During initialization, the loader reads the scenario YAML file via ROS 2’s parameter system, declaring and validating agent-specific parameters including IDs, initial poses, kinematic limits (velocity, radius), behavior profiles, Social Force Model force factors, visual appearance (skin ID), group affiliations, and goal ID sequences. A global goal registry maps goal IDs to (x, y) coordinates, enabling agents to reference shared waypoints. The loader enforces parameter constraints—Social Force Model factors are clamped to validated ranges—and supports three configuration modes: default (standard SFM values), random-normal (sampled from normal distributions for behavioral diversity), and custom (user-specified values bypass validation).

Configuration data is exposed via the `/get_parameters` ROS 2 service, returning consolidated scenario information to client nodes. The HuNav Manager invokes this service during startup with timeout-protected synchronous requests, retrieving all agent parameters, global goals, and simulation metadata (map name, simulator type, YAML base name). This service-based decoupling ensures the loader completes YAML parsing and validation before the manager begins behavior tree initialization, eliminating race conditions and parameter inconsistencies. Diagnostic logging (via `RCLCPP_INFO`) reports all loaded parameters, facilitating debugging and verification of scenario configurations. By centralizing parameter management with validation, range enforcement,

and service-based distribution, the loader supports reproducible experiments through version-controlled YAML definitions.

2.4 HuNav Evaluator and configuration tools

The HuNav RViz 2 Panel and HuNav Evaluator complete the system architecture. The RViz 2 panel generates configuration YAML files (consumed by the loader) and behavior tree XML files (executed by the manager), enabling a fully visual design-to-execution workflow. Users can create scenarios through map-based agent placement, compose behaviors via wizard interfaces or LLM-driven natural language generation, and launch Groot 2 for graphical tree editing—all integrated within a single RViz 2 interface.

The HuNav Evaluator operates as a Python ROS 2 node subscribing to `/human_states` and `/robot_states`, accumulating trajectory data controlled by service calls (`/hunav_start_recording` with experiment metadata, `/hunav_stop_recording` triggering batch computation). Metrics are implemented as functions in a modular registry (`hunav_metrics.py`), invoked based on `metrics.yaml` configuration managed through the metrics selection panel. Results are written to CSV files with dual granularity: aggregate per-experiment metrics and per-timestep trajectories. Optional map-based metrics (surprise, visibility) conditionally request occupancy grids via `/map_server/Map`.

This modular architecture supports independent component extension. Simulator wrappers implement standardized service interfaces (`compute_agents.srv`, `move_agent.srv`). Behavior trees are authored through multiple pathways (visual editor, wizard composition, LLM generation). Metrics extend by adding functions to the registry module and updating configuration files—no core node modification required. ROS 2 interfaces enable distributed deployment, external system integration via topic remapping, and hardware-in-the-loop experiments with physical robots navigating among simulated agents. This positions *HuNavSim 2.0* as a flexible platform for algorithm development and controlled validation studies in human-robot interaction.

3 HuNavSim distribution and installation

HuNavSim 2.0 is distributed as a modular set of ROS 2 packages, officially validated on the **Humble Hawksbill** distribution. The framework is designed to be deployed either via containerization or through native compilation, depending on the specific development needs.

3.1 Hardware and software requirements

The system requirements are largely dictated by the computational demands of the underlying simulation engine rather than the HuNavSim control logic. However, several core dependencies must be satisfied to ensure proper operation.

3.1.1 Software Dependencies

- **Operating System:** Ubuntu 22.04 LTS (Jammy Jellyfish) is the target platform, providing native support for ROS 2 Humble and ensuring compatibility with all system-level dependencies.
- **ROS 2 Humble Hawksbill:** The middleware layer provides the communication infrastructure, build system (colcon), and navigation stack (Nav2) required for all system components.
- **Social Force Model:** The LightSFM library¹ is mandatory for realistic human navigation behavior simulation. This library implements the Social Force Model and its extensions for group dynamics, forming the computational core of agent movement calculations.
- **Behavior Tree Framework:** Two behavior tree libraries are required for advanced agent behavior management:
 - BehaviorTree.CPP² - Core behavior tree execution engine
 - BehaviorTree.ROS2³ - ROS 2 integration layer

These libraries enable sophisticated, configurable navigation behaviors through XML-defined behavior trees.

- **People Messages:** The `people_msgs` package⁴ provides standardized message definitions for representing human agents within the ROS 2 ecosystem.

For native workspace installation, all dependencies must be manually installed and properly configured. However, containerized deployment (detailed in Section 3.2) pre-packages these dependencies, significantly simplifying the setup process.

3.1.2 Hardware Requirements

- **Base Simulator Resources:** Hardware specifications—particularly GPU capabilities and CPU core count—directly impact simulation performance and the maximum number of agents simulatable in real-time. These requirements vary significantly depending on the selected base simulator (Gazebo Classic, Gazebo Fortress, Webots, or NVIDIA Isaac Sim).
- **CPU:** Multi-core processors are recommended to support concurrent ROS 2 callback handling and allow the physics simulator to run independently alongside agent behavior computations. The Social Force Model calculations scale with the number of agents, making CPU performance critical for large-scale scenarios.

¹<https://github.com/robotics-upo/lightsfm>

²<https://github.com/BehaviorTree/BehaviorTree.CPP>

³<https://github.com/BehaviorTree/BehaviorTree.ROS2>

⁴<https://github.com/wg-perception/people/tree/ros2>

- **GPU:** Graphics processing capabilities vary by simulator:
 - Gazebo Classic and Webots: Moderate GPU requirements for visualization
 - Gazebo Fortress: Enhanced rendering capabilities with higher GPU demands
 - NVIDIA Isaac Sim: Demands an NVIDIA RTX GPU (RTX 3070 or higher) to support real-time ray tracing and physics simulation.
- **Memory:** Minimum 8 GB RAM recommended, with requirements scaling based on scenario complexity and agent count. Large-scale simulations with numerous agents and detailed environments may require 16 GB or more.

3.2 Docker containers

To ensure environment consistency and facilitate deployment, containerization is the **recommended** method for using *HuNavSim*. A dedicated repository is provided for container management.⁵

These containers pre-package the necessary system dependencies, ROS 2 middleware, and simulator wrappers. Dedicated Dockerfiles and execution scripts are provided for each supported simulator:

- **Gazebo Classic:** `gazebo_classic/Dockerfile.hunav_gz_classic11.pmb2`
- **Gazebo Fortress:** `gazebo_fortress/Dockerfile.hunav_gz_fortress`
- **NVIDIA Isaac Sim:** `isaac_sim/Dockerfile.hunav_isaac`
- **Webots:** `webots/Dockerfile.hunav_webots`

Each simulator directory includes an automated installation script (`install.sh`), an entrypoint configuration for container initialization, and a convenience run script (`run-*.bash`) that handles container execution with appropriate parameters. Comprehensive instructions for building or pulling pre-built images corresponding to each specific simulator are available in the `v2.0` branch of the repository.

3.3 Installation steps

HuNavSim 2.0 provides a Docker-based installation system that automates the setup of all dependencies, simulators, and the ROS 2 workspace. This approach eliminates manual dependency management and ensures a consistent deployment environment across different systems.

⁵https://github.com/robotics-upo/hunavsim_containers

3.3.1 Prerequisites

Before proceeding with the installation, the following software must be installed on the host system:

- **Docker Engine:** Follow the official installation guide at <https://docs.docker.com/desktop/setup/install/linux/>
- **Git:** Required to clone the HuNavSim repositories
- **NVIDIA Container Toolkit** (only for Isaac Sim option): Required for GPU-accelerated simulation. Installation guide: <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>
- **Isaac Sim Prerequisites** (only for Isaac Sim option): Complete the Container Setup steps from https://docs.isaacsim.omniverse.nvidia.com/latest/installation/install_container.html

3.3.2 Automated Installation Process

The installation is performed through an interactive script that guides users through the setup process:

1. **Clone the HuNavSim Containers Repository:**

```
git clone https://github.com/robotics-upo/hunavsim_containers.git
cd hunavsim_containers
```

2. **Execute the Installation Script:**

```
chmod +x install.sh
./install.sh
```

3. **Select Simulation Environment:** The script presents four installation options (see Figure 2):

- (a) HuNavSim 2.0 + Gazebo Classic 11 + ROS 2 Humble + PAL PMB2 robot
- (b) HuNavSim 2.0 + Gazebo Fortress + ROS 2 Humble
- (c) HuNavSim 2.0 + Isaac Sim + ROS 2 Humble + Carter robot
- (d) HuNavSim 2.0 + Webots + ROS 2 Humble + PAL TIAGo robot

4. **Automated Setup:** After selection, the script automatically:

- Builds the Docker container image with all required dependencies

- Creates a shared workspace directory (e.g., `hunav_gz_classic_ws`)
- Clones the HuNavSim core repository (`hunav_sim`)
- Clones the corresponding simulator wrapper repository
- Configures the ROS 2 workspace structure

```

HuNavSim 2.0 Docker System
Please select which HuNavSim system you want to install:

1. HuNavSim 2.0 + Gazebo Classic 11 + ROS 2 Humble + Nav2 PAL PHB2 robot
2. HuNavSim 2.0 + Gazebo Fortress + ROS 2 Humble
3. HuNavSim 2.0 + Isaac Sim + ROS 2 Humble + Nav2 Carter robot
4. HuNavSim 2.0 + Webots + ROS 2 Humble + Nav2 Tlago robot

Please select an option (1-4): 3
You selected option 3: HuNavSim 2.0 + Isaac Sim + ROS 2 Humble + Nav2 Carter robot
Installing HuNavSim docker system...
Building the docker image...
=====
[+] Building 2.5s (5/33)                                docker:default
=> [internal] load build definition from Dockerfile.hunav.isaac 0.0s
=> => transferring dockerfile: 15.15kB 0.0s
=> [internal] load metadata for nvcr.io/nvidia/isaac-sim:4.5.0 2.2s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [internal] preparing inline document 0.0s
=> [15/28] ADD https://s3.us-west-1.amazonaws.com/download.behaviortree.dev/groot2_linux_installer/Groot2-v1.6.1-x86_64 0.2s
=> [ 1/28] FROM nvcr.io/nvidia/isaac-sim:4.5.0@sha256:c2f47dc82a7714af08d376eefes8ac9d084c2b37b5d0dfbd074797ec56398fc7 0.0s

```

Figure 2: Docker installation script interface showing the interactive menu and automated build process for the Isaac Sim option. The script guides users through simulator selection and handles all dependency installation automatically.

The Docker containers automatically install and configure all software dependencies listed in Section 3.1.1.

The shared workspace structure ensures that users can modify scenarios, create new environments, and access source code from the host system while maintaining isolation of system dependencies within the container.

4 Execution of predefined scenarios

A critical aspect of *HuNavSim 2.0* is its ability to operate across multiple simulation backends, ensuring that social navigation algorithms can be benchmarked under varying visual and physical fidelities. The framework includes a suite of pre-configured scenarios distributed across the supported simulator wrappers, accessible through both containerized and native installations.

4.1 Available Scenarios

The system provides a diverse set of environments, ranging from domestic to industrial settings, designed to test different aspects of human-robot interaction (HRI) and navigation under varying levels of confinement and visual complexity. Each scenario is pre-configured with specific agent populations, behavioral profiles, and navigation goals. These scenarios are categorized by the underlying simulator, as illustrated in Figure 3.

- **Gazebo Classic:** Environments optimized for computational efficiency while maintaining physical accuracy for sensor simulation (e.g., LiDAR ray-casting). The PAL Robotics PMB2 differential-drive robot is used across all scenarios, equipped with Nav2 for autonomous navigation.
 - *House (3 agents):* A multi-room domestic topology comprising a bedroom, living room, kitchen and a gym area with obstacles such as weights and benches. Three agents with Regular behavior navigate cyclic routes through different rooms, testing service robot performance in cluttered, non-convex spaces.
 - *Warehouse (3 agents):* A logistics environment characterized by high vertical shelving and narrow aisles. Three Regular agents patrol between loading zones and storage areas, suitable for testing path planning and corner-turning behaviors in constrained industrial corridors.
 - *Cafe (3 agents):* A social space featuring service and dining zones. Agent population includes Regular and social behaviors (Surprised, Threatening), simulating varied customer interactions in mixed-use environments with navigation between seating areas and service counters.
- **Gazebo Fortress:** Modern indoor environments leveraging the next-generation Gazebo simulator. Currently, it operates without a physical robot model, focusing on agent-only simulations. To include a robot is an ongoing work.
 - *Cafe (3 agents):* A detailed social space featuring two distinct zones (a service area with kitchen counters and tiled flooring, and a dining area with wooden flooring and tables), ideal for testing social navigation algorithms in mixed-use social environments without robot interference.
- **Webots:** Scenarios leveraging efficient rendering with distinct functional zones. The PAL Robotics TIAGo Lite mobile manipulator is deployed, featuring omnidirectional navigation capabilities.
 - *Office (2 agents):* A detailed corporate workspace featuring wooden flooring and a structured layout with two central islands of desks, side lockers, and a lounge area with sofas. Two Regular agents navigate between workstations and common areas, simulating realistic office traffic flows.
 - *Factory (4 agents):* An industrial environment with concrete flooring, populated by static obstacles such as stacks of wooden pallets and metal barrels. Agent population includes diverse behaviors: Regular, Threatening, Surprised creating dynamic interaction scenarios for testing navigation around typical industrial clutter.

- **NVIDIA Isaac Sim:** Photorealistic environments utilizing the Omniverse platform with advanced physics (PhysX), enabling the testing of vision-based navigation systems. Multiple robot platforms are supported, including the NVIDIA Carter differential-drive robot with Nav2 integration. All pre-configured scenarios feature Regular behavior agents, though behaviors can be easily modified either through the HuNavSim RViz2 panel interface or by editing the corresponding YAML configuration files.
 - *Warehouse Digital Twin (5 agents):* A high-fidelity replica of a logistics center, featuring complex geometry such as forklifts, pallet jacks, and extensive shelving systems. Five agents navigate between storage zones and loading docks.
 - *Hospital (9 agents):* A detailed clinical environment with corridors equipped with reception desks, waiting areas, wheelchairs, and medical carts. Ten agents navigate between corridors and public areas, providing a context-rich scenario for healthcare robotics testing with realistic pedestrian flows.
 - *Large Office (8 agents):* A comprehensive corporate environment combining a spacious entrance hall with elevators and reflective flooring with extensive office areas. Ten agents simulate employee movements between workspaces, meeting rooms, and common facilities, testing navigation in architecturally complex spaces.

4.2 Launching Scenarios

Scenarios can be executed through either the Docker-based system (recommended for most users) or native ROS 2 installations for development workflows.

4.2.1 Docker-Based Execution

The Docker run scripts provide an interactive interface for scenario selection. After installation completes, the system can be launched using the provided run script:

```
cd <simulator_directory>
chmod +x run-<simulator>.bash
./run-<simulator>.bash
```

For Gazebo Classic, Gazebo Fortress, and Webots, the run script launches an interactive menu displaying available scenarios and configuration options (see Figure 4). Users can select from pre-configured scenarios stored in the **scenarios** directory, create new environments using RViz, or access a bash shell for manual operations. For Isaac Sim, the script prompts for Omniverse credentials on first run and launch the Isaac Sim container. The container handles workspace compilation and ROS 2 environment configuration automatically.



(a) Gazebo Classic: House



(b) Gazebo Classic: Warehouse



(c) Gazebo Fortress: Cafe



(d) Webots: Office



(e) Isaac Sim: Hospital



(f) Isaac Sim: Warehouse

Figure 3: Visual overview of the diverse simulation environments provided by *HuNavSim 2.0* across different platforms. The scenarios cover domestic (House), commercial (Cafe, Office), and industrial (Warehouse/Factory) settings.

4.2.2 Native ROS 2 Execution

For users working with a native source installation, scenarios are launched using standard ROS 2 launch commands. The workspace must be sourced prior to execution:

```
source install/setup.bash
```

```
===== HuNavSim Docker Menu =====
1) Run scenario agents_cafe.yaml
2) Run scenario agents_house.yaml
3) Run scenario agents_warehouse.yaml
4) Create a new scenario with RViz
5) Open a bash shell
6) Exit container
=====
Select an option (number): █
```

Figure 4: Interactive menu displayed by the Docker run script for Gazebo Classic. The menu shows available scenario files and options for environment creation. Users specify a scenario by its numeric identifier, prompting the system to initiate the corresponding simulation.

Gazebo Classic

Launch files are provided for different robot/world combinations. To launch the Cafe scenario with a static actor:

```
ros2 launch hunav_gazebo_wrapper example_cafe.launch.py
```

To launch with a real robot (e.g., PMB2):

```
ros2 launch hunav_gazebo_wrapper pmb2_cafe.launch.py
```

Gazebo Fortress

Launch the Cafe environment populated with *HuNavSim* agents:

```
ros2 launch hunav_gazebo_fortress_wrapper cafe_fortress_test.launch.py
```

Webots

Distinct launch files are provided for each scenario. To launch the Factory environment:

```
ros2 launch hunav_webots_wrapper hunavsim_launch_factory.py
```

Similarly, `hunavsim_launch_office.py` and `hunavsim_launch_hall.py` are available for the Office and Hall scenarios.

NVIDIA Isaac Sim

The interaction with the Omniverse platform is managed through a dedicated launcher node:

```
ros2 run hunav_isaac_wrapper hunav_isaac_launcher
```

This launcher provides an interactive CLI for selecting the specific scenario (Warehouse, Hospital, Large Office), robot model, and *HuNavSim* agent configuration parameters before loading the environment.

5 Creation of new scenarios

While *HuNavSim 2.0* provides a suite of pre-configured scenarios, a core objective of the framework is to facilitate the creation of custom environments and agent behaviors. To address the complexity of manually authoring simulation files (SDF) and behavior logic (Behavior Tree XMLs), the system incorporates a set of assisted generation tools driven by Large Language Models (LLMs) and automated mapping utilities.

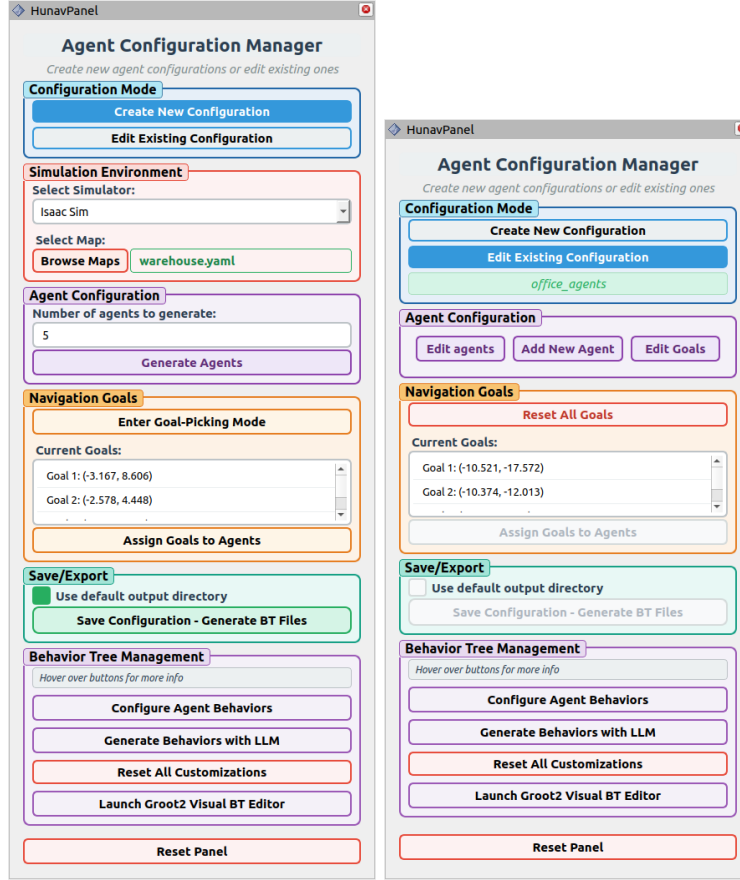
5.1 RViz panel for scenario generation

HuNavSim 2.0 implements a comprehensive graphical interface for scenario design through a dedicated RViz 2 panel, providing a *what-you-see-is-what-you-simulate* workflow. The panel integrates agent configuration, goal assignment, and behavior tree management into a unified visual environment, enabling rapid progression from map setup to complete simulation scenarios without requiring manual file editing.

Configuration modes and simulator selection The panel operates in two distinct modes: *Create Mode* for generating new scenarios from scratch, and *Edit Mode* for modifying existing configurations (see Figure 5). Upon initialization, the user selects the target simulator back-end—Gazebo Classic, Gazebo Fortress, Isaac Sim, or Webots—which determines simulator-specific parameters such as agent spawn heights and available visual appearances. The simulator selection triggers the map loading interface, allowing any 2D occupancy map compatible with `nav2_map_server` to be loaded and visualized within RViz 2.

Interactive agent creation and configuration Agent creation proceeds through a sequential dialog system that combines parametric configuration with in-scene spatial placement. For each agent, the interface exposes the core motion parameter (maximum velocity), behavior profiles (regular, impassive, curious, scared, surprised, threatening), and Social Force Model parameters (see Figure 6). Users may retain default values, sample from predefined distributions, or manually adjust individual parameters. A simulator-specific appearance catalogue provides visual diversity across the agent population. Spatial placement is achieved through an interactive tool that captures mouse clicks on the loaded map, setting both position and orientation via a click-and-drag mechanism. The system immediately visualizes each configured agent with color-coded markers and orientation indicators, with the interface automatically advancing to the next agent until the specified population is complete.

Goal definition and assignment workflow Following agent placement, the panel provides a dedicated goal management system. Activating *Goal-Picking Mode* enables direct map interaction: each mouse click creates a numbered goal marker, which is stored in a global goal registry and displayed in a scrollable list.



(a) Create Mode

(b) Edit Mode

Figure 5: *HuNavSim* RViz 2 panel operational modes: (a) Create Mode for generating new scenarios with sequential agent population and goal definition, and (b) Edit Mode for modifying existing configurations with direct access to agent parameters, goal reassignment, and behavior tree customization.

Goals may be repositioned by clicking near an existing marker and selecting a new location. The *Assign Goals to Agents* dialog presents a dual-list interface wherein users associate specific goals with individual agents through an intuitive assignment mechanism (see Figure 7). As goals are assigned, the system computes and visualizes navigation paths directly in the RViz 2 environment, providing immediate feedback on agent navigation patterns. The interface validates that all agents receive at least one goal before allowing progression.

Configuration persistence and iteration The *Save Configuration* operation serializes the complete scenario to a YAML file (containing agent param-

Agent 1 of 5

Agent 1 of 5

Basic Configuration

Maximum velocity (m/s):
1.5

☒ Cyclic navigation

Behavior Configuration

Behavior type:
Regular

Configuration preset:
Default

Simulator Options

Agent appearance:
Random

SFM parameters

Goal Force Factor:
10

Obstacle Force Factor:
2

Social Force Factor:
5

Set Initial Pose on Map

Next agent

Cancel

Figure 6: Sequential agent configuration dialog providing comprehensive parameter specification: basic motion constraints (maximum velocity, navigation mode), behavior profile selection with preset configurations, simulator-specific appearance options, and Social Force Model parameters (goal, obstacle, and social force factors). The interactive spatial placement tool enables direct positioning on the loaded map.

ters, goal assignments, and spatial configurations) and auto-generates one XML behavior tree per agent. Both file types are written to the target simulator wrapper’s `scenarios` and `behavior_trees` directories, respectively, with filenames derived from the map name and scenario identifier. In Edit Mode, opening an existing YAML file repopulates the panel with all agents, goals, paths, and behavior configurations, enabling point-and-click modification of any scenario element. This bidirectional workflow supports rapid iteration, parameter sweeps, and focused ablation studies while maintaining complete traceability between visual configurations and persistent scenario definitions.

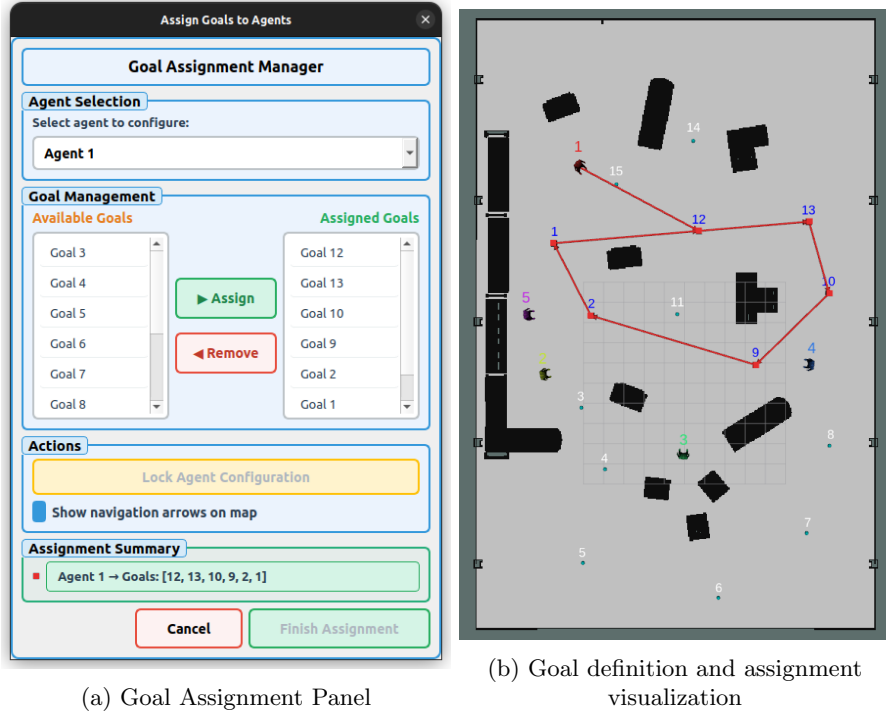


Figure 7: Interactive goal management interface: (a) Goal Assignment Manager dialog featuring dual-list selection, agent-specific configuration, and assignment validation with real-time summary feedback; (b) RViz 2 visualization of defined goals (numbered markers in cyan) with assigned navigation sequences rendered as colored paths connecting agent initial poses to their target waypoints.

Behavior tree management In addition to selecting a specific behavior type for each agent at configuration time, the panel also incorporates three distinct yet complementary methodologies for specifying behavior trees. First, a *Configuration Wizard* provides a structured, multi-page dialog for constructing agent behaviors from predefined functional blocks—including navigation sequences, social interaction patterns, and environmental responses. Users select blocks from a categorized library, assign them to individual agents or groups, configure block-specific parameters, and preview the resulting tree structure before finalization. Second, a *LLM generator* enables natural-language behavior specification: users provide textual descriptions of desired agent behaviors, and the system synthesizes corresponding behavior trees through an integrated LLM interface. Third, direct integration with *Groot 2* allows graphical inspection and manual modification of generated trees. All three methods operate on the same underlying XML behavior tree format, ensuring consistency and interoperability.

The RViz 2 panel architecture thus transforms scenario creation from a code-intensive task to an interactive visual process, while preserving the flexibility of YAML and behavior tree representations. Complete documentation and usage examples are maintained in the `hunav_rviz2_panel` repository.

5.2 Assisted generation of human behaviors

The creation of realistic human behaviors in *HuNavSim 2.0* relies on Behavior Trees (BTs) implemented using the *BehaviorTree.CPP* library. While the system provides a comprehensive node library for expressing complex social interactions, manually designing behavior trees—particularly for multi-agent scenarios with diverse intentions and coordinated activities—remains time-intensive and error-prone. To address these challenges, *HuNavSim 2.0* introduces two complementary approaches for assisted behavior specification: a visual wizard interface for composing behaviors from predefined functional blocks, and an LLM-based generator for natural language-driven behavior synthesis.

5.2.1 Visual behavior composition wizard

The `BTConfigDialog` wizard, accessible via the *Configure Agent Behaviors* button in the RViz 2 panel’s Behavior Tree Management section, provides a multi-page guided interface for constructing agent behaviors without manual XML editing. The wizard implements a three-stage workflow that progressively builds complete behavior configurations through intuitive visual interactions.

The first stage, *Block Selection and Assignment* (Figure 8), presents users with a categorized library of ten functional behavior blocks in the left panel. These blocks encompass four behavioral categories: robot interaction patterns (Robot Interaction, Robot Awareness, Robot Blocking), social behaviors and environmental responses (Social Interaction, Audio-Triggered Social, Attention Response, Friendly Greeter), agent coordination (Follow Agent, Group Coordination), and protective behaviors (Guardian Protector). Each block displays a descriptive subtitle explaining its purpose, such as “Detect, look at, and approach robot” for the Robot Interaction block. The right panel features an interactive assignment matrix where users specify which behaviors each agent should execute by toggling checkboxes at the intersection of behavior rows and agent columns. Bulk operations are supported through the “Assign All” and “Clear All” buttons, enabling rapid configuration of homogeneous agent groups or complete reconfiguration when experimenting with different behavioral compositions.

The second stage, *Per-Agent Configuration* (Figure 9), provides fine-grained control over individual agent behaviors. A dropdown selector at the top allows users to switch between agents, with each agent’s configuration displayed independently. The left panel presents the agent’s assigned blocks as movable elements in a vertical execution order list, where users can reorder blocks using “Up”, “Down”, and “Shuffle” buttons to control the behavior tree’s sequential execution flow. Also, the wizard supports randomization through a checkbox

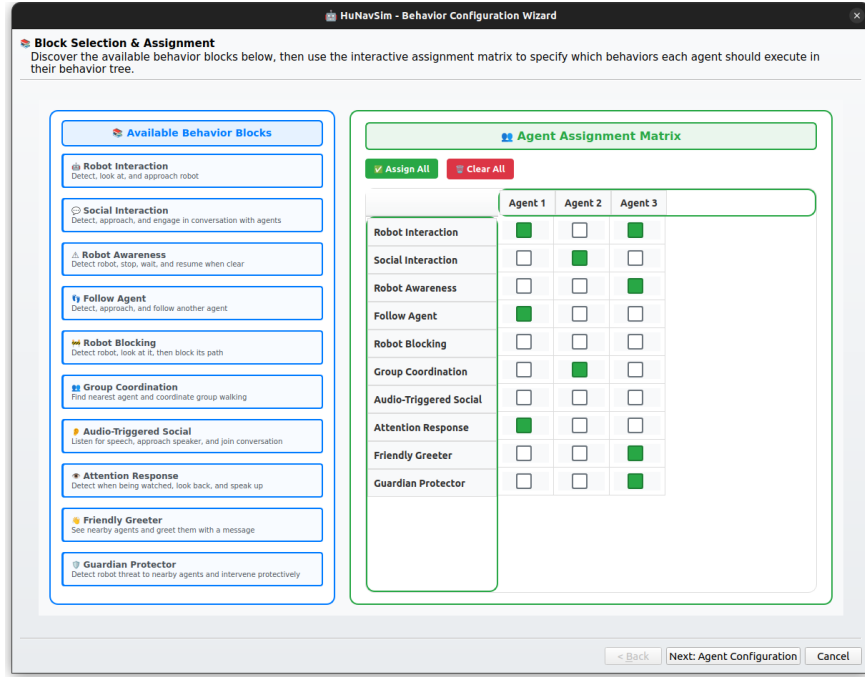


Figure 8: Block Selection and Assignment stage. The left panel displays ten categorized behavior blocks (robot interaction, social behaviors, agent coordination, environmental responses, and protective behaviors). The right panel features an interactive assignment matrix where users toggle checkboxes to assign behaviors to individual agents, with “Assign All” and “Clear All” buttons for bulk operations.

that enables probabilistic block execution, allowing designers to create more varied and naturalistic agent behaviors. The right panel displays a node execution mode selector (Always Execute, Execute Only Once, or Random Execution with probability) and dynamically generates parameter input widgets based on the selected block’s requirements. For instance, the Robot Interaction block exposes four configurable parameters: Robot Detection Distance (default 5.0m), Robot Approach Distance (default 1.50m), Maximum Approach Speed (default 1.80 m/s), and Robot Interaction Duration (default 30.0s). These parameters directly map to behavior tree node ports, with validation ensuring that distance thresholds maintain logical relationships (detection distance must exceed approach distance) and duration values remain within reasonable bounds.

The third stage, *Preview and Validation* (Figure 10), consolidates the configuration into two synchronized views. The upper summary panel displays a compact table showing each agent’s complete behavior sequence as a visual chain of block icons and labels (e.g., “Robot Interaction → Robot Awareness → Friendly Greeter → Guardian Protector” for Agent 3), enabling quick verifica-

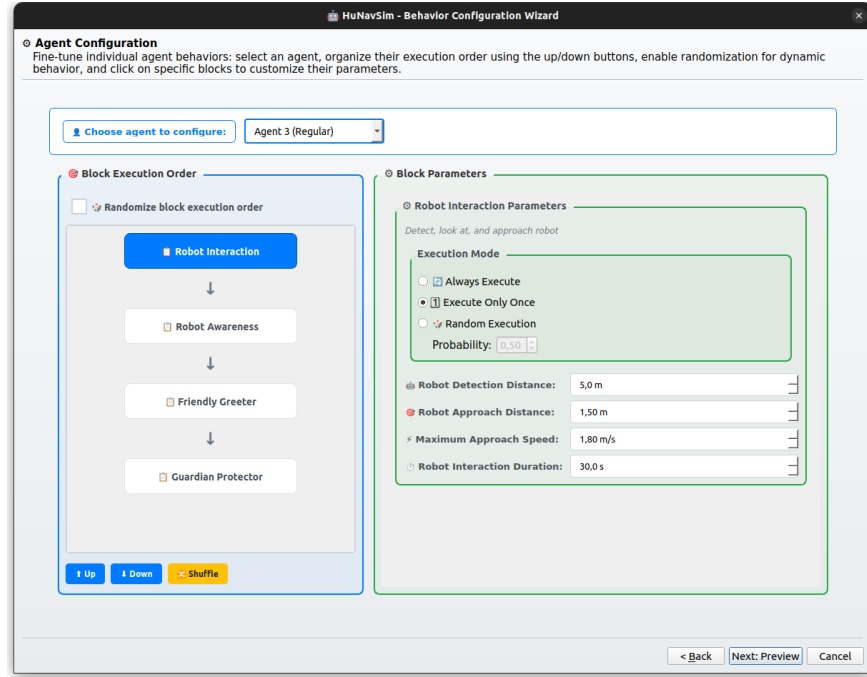


Figure 9: Per-Agent Configuration stage for Agent 3 (Regular behavior type). The left panel shows assigned blocks in execution order with drag-and-drop re-ordering controls (Up, Down, Shuffle buttons). The right panel dynamically generates controls for the selected block, here showing Robot Interaction parameters including execution mode, detection distance (5.0 *m*), approach distance (1.50 *m*), maximum approach speed (1.80 *m/s*), and interaction duration (30.0 *s*).

tion of inter-agent behavioral diversity. The lower XML preview panel presents the generated behavior tree code that will be injected into each agent’s XML file. Users can select different agents from a dropdown to examine their specific XML patches, which include complete `<Sequence>` elements with parameterized `<RunOnce>`, `<Condition>`, and `<Action>` nodes. For example, the EngageRobotBlock sequence shown in the preview demonstrates the structure: initial visibility checks (`IsRobotVisible`), proximity detection (`IsRobotClose`), approach actions (`ApproachRobot`), and movement resumption (`ResumeMovement`), with agent-specific IDs and thresholds automatically populated from the configuration stage. An “Apply to Agents” button commits the changes, triggering the XML patching process.

The wizard maintains internal state through Qt data structures mapping agent indices to block assignments and parameter dictionaries. Block definitions specify required input ports (e.g., `agent_id`, `time_step`, `threshold`), enabling the wizard to generate appropriate parameter input fields dynam-

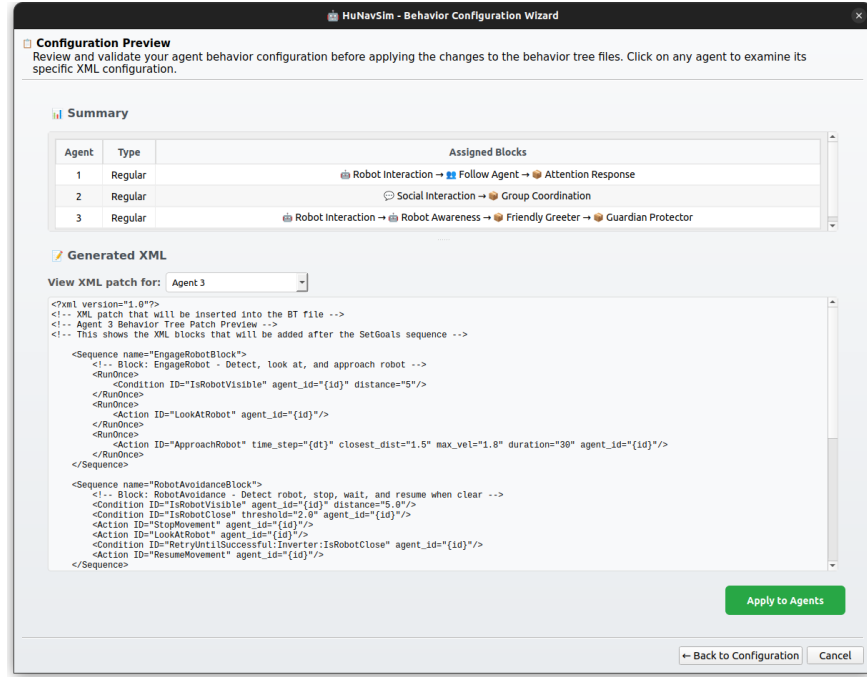


Figure 10: Preview and Validation stage. The upper summary table displays each agent’s complete behavior sequence as icon chains (e.g., Agent 3: Robot Interaction → Robot Awareness → Friendly Greeter → Guardian Protector). The lower panel shows the generated XML code with agent-specific sequences including parameterized conditions and actions. Users can select different agents via dropdown to examine their specific XML patches before applying changes via the green button.

ically through the `createBlockParametersForAgent()` method, which constructs `QSpinBox`, `QDoubleSpinBox`, `QComboBox`, and `QCheckBox` widgets based on parameter types. Upon completion, the `patchAllAgentBTFiles()` method modifies existing behavior tree XML files by injecting wizard-configured sequences into template-generated trees through precise XML manipulation using the `TinyXML2` library. The patching operation locates the `SetGoals` sequence anchor within each agent’s tree and inserts the custom behavior blocks immediately after, preserving the original file structure and comments while enabling iterative refinement. The wizard can load existing configurations from XML files via `loadExistingConfiguration()`, which parses agent BT files, identifies behavior sequences through pattern matching against known block signatures, and extracts parameter values from XML attributes to populate the wizard’s interface, enabling round-trip editing workflows where users can visually modify previously generated or manually edited trees.

This visual composition approach reduces the barrier to entry for users unfa-

miliar with BehaviorTree.CPP syntax while maintaining full compatibility with manually edited or LLM-generated trees. The wizard’s validation layer, implemented in `validateConfiguration()`, enforces consistency constraints such as ensuring each agent has at least one assigned block, verifying parameter values fall within valid ranges, and checking for goal conflicts when agents share behavioral objectives. Error states are communicated through styled warning messages in the validation feedback panel, guiding users toward correct configurations before XML generation proceeds.

5.2.2 LLM-based behavior generation

For users preferring natural language specification or requiring behaviors beyond the wizard’s predefined block library, the `hunav.behavior.tree.generator` package provides automated generation through large language models. Accessible via the *Generate Behaviors with LLM* button in the RViz 2 panel, this Python-based ROS 2 tool launches an interactive terminal session that guides users through behavior description and generation. The generator operates in two modes: *scenario mode* (generating behaviors for loaded agent configurations with pre-defined goals) and *manual mode* (creating behaviors from scratch with user-specified agent counts and goal positions). The RViz 2 integration uses scenario mode, automatically passing the current YAML file path and simulator type as command-line arguments.

The generation pipeline comprises four modular components: Scenario Manager parses YAML files to extract agent identifiers, initial poses, kinematic constraints, and assigned goal sequences, normalizing configuration data across simulator wrappers; Behavior Manager constructs agent-specific prompts by combining user-provided natural language descriptions with scenario context, invoking the LLM and post-processing responses to extract valid XML content by detecting closing tags and truncating extraneous text; LLM Manager abstracts model communication through OpenAI-compatible APIs, supporting both cloud services (OpenAI GPT) and locally-hosted alternatives (Ollama, LM Studio), loading system prompts from template files that specify HuNavSim’s node library, XML constraints, BehaviorTree.CPP syntax rules, and few-shot examples; File Generator orchestrates multi-agent workflows, assembling complete XML documents by combining template headers and footers with LLM-generated behavior sections, stripping markdown artifacts, and saving files with deterministic naming conventions to the simulator wrapper’s `behavior_trees` directory.

5.2.3 Comparative workflow and integration

The wizard and LLM approaches serve complementary use cases: the wizard excels at rapid prototyping with known behavior patterns, providing immediate visual feedback and ensuring syntactic correctness through constrained composition, while the LLM generator enables open-ended specification of novel behaviors, supporting complex temporal logic and conditional branching that may not

fit predefined block templates. Both methods produce XML files consumable by the HuNav Manager’s *BehaviorTree.CPP* executor, enabling mixed-mode workflows where base behaviors are wizard-generated and subsequently refined through LLM iteration or manual editing in Groot 2.

The RViz 2 panel’s *Reset All Customizations* button provides a unified mechanism to revert all agents to default template-based behaviors, enabling clean experimentation cycles regardless of the original authoring method. The *Launch Groot2 Visual BT Editor* button opens generated trees in Groot 2 for graphical inspection, supporting verification and manual refinement. This multi-modal behavior authoring architecture accommodates diverse user expertise levels and workflow preferences, from novice users leveraging wizard-guided composition to experienced developers employing LLM-assisted rapid prototyping or direct XML manipulation, all within a unified RViz 2 interface that maintains consistency between visual configuration, behavior specification, and simulation execution.

5.3 Assisted Gazebo environment generation

Creating diverse and functional simulation environments is traditionally a manual, labor-intensive task involving syntax-heavy file authoring in Simulation Description Format (SDF). To address this, the system incorporates the `gazebo_world_generator` package and auxiliary tools to automate the production of physics-enabled 3D worlds through two distinct input modalities.

5.3.1 Using natural language

The primary method leverages Large Language Models within a hybrid neuro-symbolic architecture to transform unstructured natural language descriptions (e.g., "An 8m x 6m office with 4 desks and a meeting area") into collision-free, navigable Gazebo worlds. This pipeline implements a multi-phase workflow comprising nine integrated stages.

Configuration: Before using the natural language generator, users must configure an OpenAI-compatible LLM endpoint. The system supports multiple backends including OpenAI API, LM Studio, Ollama, or any custom OpenAI-compatible server.

Configuration is performed by editing `config/generator_config.yaml`:

```
llm:
  server_url: "http://localhost:1234/v1" # LLM server endpoint
  model_name: "your-model-name"        # Model identifier
```

For OpenAI API users with an API key, set the server URL to `https://api.openai.com/v1` and configure the model name (e.g., `gpt-4`). The API key should be set as an environment variable or included in the configuration file. For local deployments using LM Studio or Ollama, specify the local endpoint address

and the loaded model name. The configuration file also allows customization of placement parameters such as minimum object distance (default: 0.5m) and corridor width (default: 1.2m).

Execution: The generator can be launched in two modes. Interactive mode presents a menu-driven interface for creating new worlds, refining existing ones, or testing the LLM connection:

```
ros2 run gazebo_world_generator generate_world
```

Command-line mode allows direct world generation with explicit parameters:

```
ros2 run gazebo_world_generator generate_world \  
  --description "12m x 10m warehouse with 8 storage racks" \  
  --output custom_warehouse.sdf
```

Available command-line parameters include: `--description` (required natural language world specification), `--output` (custom output filename; defaults to timestamped name if omitted), `--llm-server` (override configured LLM endpoint), `--model` (override configured model name), and `--debug` (enable verbose logging for troubleshooting). The system accepts dimension specifications in multiple formats including "12m x 10m office", "warehouse (20x15m)", or "office of size 8x6". Crucially, users must specify exact object quantities—the system generates only what is explicitly requested (e.g., "5 pallets" creates exactly 5 pallets, no implicit objects are added). Figure 11 shows an example office environment generated from the description "Two 10m x 8m offices connected by a corridor. First office has 6 desks and chairs. Second office has a table with 4 chairs around it".

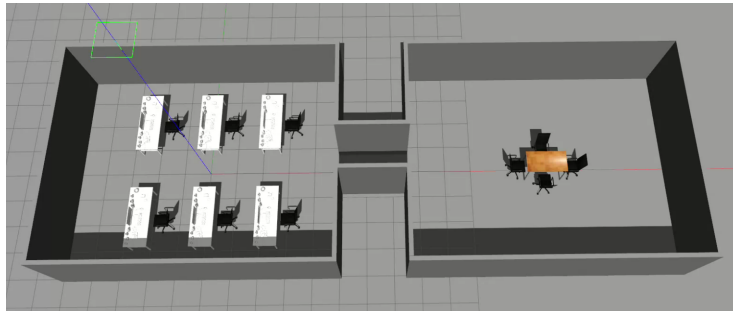


Figure 11: Example of a Gazebo world automatically generated from natural language description using the LLM-based pipeline. The environment includes office desks with chairs and a meeting area with collision-free navigation space.

Output Generation: Upon execution, the system creates a timestamped directory structure in `generated_worlds/` containing three categories of artifacts:

- `worlds/world_YYYYMMDD_HHMMSS.sdf`: The complete Gazebo world file in Simulation Description Format, including physics configuration, lighting, and all placed objects with collision geometries.
- `occupancy_maps/world_YYYYMMDD_HHMMSS.pgm/.yaml`: A 2D occupancy grid in Portable Gray Map format with accompanying YAML metadata, ready for integration with ROS 2 Nav2 navigation stack.
- `logs/world_YYYYMMDD_HHMMSS.log`: Detailed generation log including LLM interactions, model resolution decisions, and collision resolution iterations.

Processing Pipeline: The conversion operates through a nine-stage automated pipeline. First, dimension extraction and LLM parsing use structured prompts to analyze the natural language description, extracting explicit dimensions and converting unstructured text into validated JSON with self-healing retry logic for malformed responses. Second, the *SmartModelResolver* grounds abstract object names to 3D models through a three-tier strategy: LLM-based semantic matching, automatic download from online repositories (Gazebo Fuel, GitHub), and deterministic keyword matching with synonym expansion (e.g., "couch" $\leftrightarrow$ "sofa"). Third, the *NaturalPlacementEngine* combines LLM-guided reasoning for initial layouts with geometric algorithms for refinement, using a *SemanticGroupingEngine* to identify functional relationships (chairs with desks, lamps near workbenches) and supporting both semantic grouping for offices and grid layouts for warehouses. Finally, iterative validation and collision resolution enforce boundary checks, doorway clearances, and hierarchical collision detection through Pydantic-based data models, guaranteeing physics-consistent environments without overlaps. Additional stages handle object expansion, room layout optimization, world generation with physics configuration, and automatic 2D occupancy grid generation for robot navigation (see Section 5.4).

5.3.2 Using a ROS 2 map

In scenarios where a 2D occupancy grid map exists (e.g., obtained from a previous SLAM session with a real robot) but the corresponding simulation world is missing, the system provides the `pgm_to_sdf_converter` package. This tool automates the conversion of 2D occupancy grid maps (PGM/YAML format, standard ROS 2 map representation) into 3D Gazebo simulation environments.

Execution: The converter requires only the YAML map file as input and automatically locates the associated PGM image file:

```
ros2 run pgm_to_sdf_converter pgm_to_sdf map.yaml
```

The YAML file extension can be omitted for brevity. By default, the output SDF file is created in the same directory as the input map with a matching filename (e.g., `map.yaml` produces `map.sdf`).

Common Parameters: Several command-line options allow customization of the conversion process:

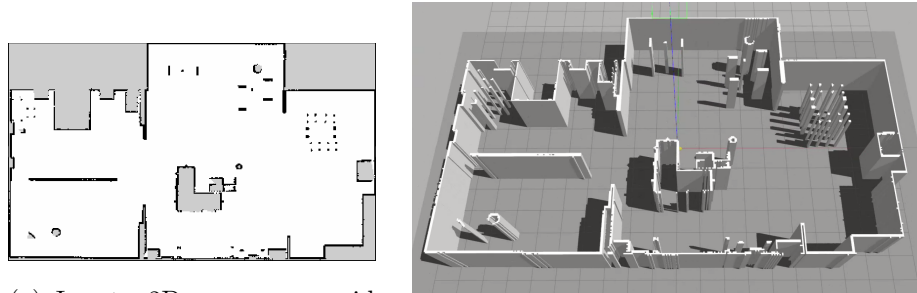
```
ros2 run pgm_to_sdf_converter pgm_to_sdf map.yaml \  
  -o custom_world.sdf \      # Custom output filename  
  --wall-height 3.0 \        # Wall height (default: 2.5 m)  
  --min-wall-length 1.0 \    # Filter segments (default: 0.5 m)  
  --keep-cleaned-pgm         # Save cleaned pgm for inspection
```

The `--wall-height` parameter controls the vertical extrusion of all detected obstacles, while `--min-wall-length` filters out short wall segments that may represent noise or small objects. The `--keep-cleaned-pgm` flag is useful for debugging, as it saves the preprocessed version of the map alongside the output SDF.

Output: Upon completion, the converter generates a complete Gazebo world file in SDF format at the specified output location. This file can be directly launched in Gazebo Classic:

```
gazebo map.sdf
```

The generated world contains optimized wall geometries with physics properties and default lighting configuration, ready for robot navigation testing. Figure 12 illustrates the conversion process from a 2D occupancy grid map of a small house to a fully navigable 3D Gazebo environment.



(a) Input: 2D occupancy grid map

(b) Output: Generated 3D Gazebo world

Figure 12: PGM-to-SDF conversion pipeline: transformation from a 2D occupancy grid map (left) into a collision-enabled 3D simulation environment (right) with extruded walls preserving the original floor plan geometry.

Processing Pipeline: The conversion operates through a four-stage pipeline that requires no user intervention. First, the system performs adaptive noise analysis by examining the map’s connected components, boundary characteristics, and pixel fragmentation to classify the noise level (Clean, Moderate, or

Heavy). Second, context-aware image enhancement applies morphological operations with parameters automatically scaled to the detected noise level—heavily noisy SLAM maps receive aggressive filtering, while clean CAD maps use conservative processing to preserve architectural details. Third, geometric extraction employs a two-pass scanning algorithm (horizontal then vertical) to identify maximal wall segments from occupied pixels, representing the map as optimized rectangles rather than individual pixels. Finally, segment optimization merges adjacent aligned walls sharing identical orientation and extent, typically reducing object count by 60-70% for improved simulation performance. Since 2D maps lack height information, all obstacles are uniformly extruded as walls (default 2.5m height), producing a functional environment suitable for navigation testing without manual modeling effort.

5.4 Creation of ROS 2 navigation maps

To enable autonomous robot navigation within generated simulation environments, the ROS 2 navigation stack (Nav2) requires a 2D occupancy grid map. Traditionally, obtaining this map involves manual robot teleoperation combined with SLAM algorithms. To streamline this workflow, *HuNavSim* integrates the `gazebo_ros2_2dmap_plugin`, which automatically generates occupancy grid maps directly from Gazebo Classic worlds without robot deployment or SLAM execution.

Automated Generation: The plugin provides a fully automated script that handles the complete map generation workflow:

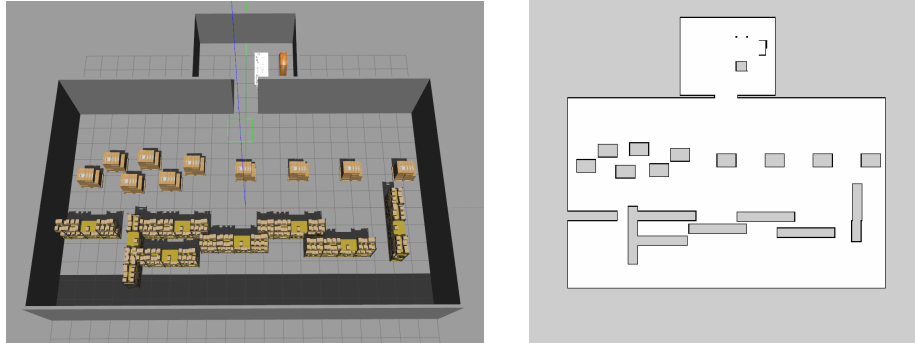
```
ros2 run gazebo_ros2_2dmap_plugin generate_map.sh world.sdf output_dir
```

This command automatically injects the plugin into the world file (if not already present), launches Gazebo in headless mode, generates the occupancy map, saves the output files (`world.yaml` and `world.pgm`), and performs cleanup. The output files follow standard ROS 2 map format, directly compatible with Nav2 without modification. Figure 13 demonstrates the automated generation of a navigation map from a complex warehouse-office hybrid environment.

Manual Generation: For users requiring custom configuration, the plugin can be manually added to world files within the `<world>` tags:

```
<plugin name='gazebo_occupancy_map' filename='libgazebo_2Dmap_plugin.so'>
  <map_resolution>0.05</map_resolution>  <!-- 5cm per cell -->
  <map_height>0.3</map_height>            <!-- slice at 30cm -->
  <!-- Omit map_size_x/y for auto-detection -->
</plugin>
```

After launching Gazebo with the world, map generation is triggered via ROS 2 service:



(a) Gazebo world with warehouse and office areas

(b) Auto-generated 2D occupancy grid map

Figure 13: Automated navigation map generation using the `gazebo_ros2_2Dmap_plugin`. The plugin extracts ground-truth occupancy information directly from the 3D world geometry (left), producing a ROS 2-compatible occupancy grid (right) ready for Nav2 navigation without SLAM or manual mapping.

```
ros2 service call /gazebo_2Dmap_plugin/generate_map \\  
std_srvs/srv/Empty  
ros2 run nav2_map_server map_saver_cli -f output_map --ros-args \\  
-r map:=map2d
```

Configuration Parameters: Key parameters include `map_resolution` (cell size in meters, default: 0.05), `map_height` (height at which to slice the world, typically 0.2-0.5m), and `map_margin` (extra space around auto-detected bounds, default: 2.0m). When `map_size_x` and `map_size_y` are omitted, the plugin automatically computes world bounds by analyzing all model bounding boxes and centers the map at the world origin (0,0), ensuring coordinate consistency between simulation and navigation frames.

Technical Operation: The map generation operates by querying 3D world geometry through the Gazebo physics engine rather than simulating sensor data. The plugin casts rays at the configured height to detect obstacle intersections, providing ground-truth occupancy information without sensor noise. Starting from the specified initial position, a wavefront expansion algorithm propagates through an 8-connected grid, systematically marking cells as free (0), occupied (100), or unknown (-1) based on ray-casting results. This approach generates standard ROS 2 maps in seconds through a single service call, ensuring that any environment created by the `gazebo_world_generator` is immediately ready for navigation experiments without additional data collection procedures.

6 Benchmarking

HuNavSim 2.0 provides an integrated benchmarking system that enables systematic evaluation of social navigation performance through a comprehensive suite of metrics. The benchmarking pipeline comprises metric selection, automated computation during simulation, and structured result generation, supporting both standardized evaluation protocols and user-defined metric extensions.

6.1 RViz panel for metrics configuration

The metrics configuration interface is implemented as a dedicated RViz 2 panel (`hunav_metrics_panel`) that provides interactive selection and management of evaluation metrics (see Figure 14). The panel organizes the 33 available metrics into six taxonomic categories: *Navigation Performance* (task completion time, path length, goal-reaching accuracy), *Social Behavior* (proxemic space intrusions, distance maintenance to pedestrians, group interaction patterns), *Motion Analysis* (velocity profiles, acceleration patterns, heading changes), *Safety & Collisions* (collision detection and classification), *Social Forces* (forces experienced by agents and robot based on the Social Force Model), and *Danger & Surprise Metrics* (fear, panic, visibility-based surprise, shock, and reaction costs).

Each metric is presented as a labeled checkbox with comprehensive tooltip documentation describing its definition, physical units, theoretical foundation (e.g., Proxemics theory for space intrusions, Helbing-Molnár Social Force Model for force-based metrics), and interpretation guidelines. A real-time search functionality enables filtering by metric name or keyword, while *Select All* and *Deselect All* controls facilitate batch configuration. The panel displays the current selection count and provides immediate visual feedback upon configuration save operations. All selections are persisted to the `hunav_evaluator` package’s `metrics.yaml` configuration file, ensuring consistency across evaluation sessions.

6.2 Metric computation during simulation

The complete architecture of the HuNavSim’s metric computation system is illustrated in Figure 15. Metric computation is performed by the HuNav Evaluator node, which subscribes to `/human_states` (agent positions and velocities published as `hunav_msgs/Agents` messages) and `/robot_states` (robot position and velocities published as `hunav_msgs/Agent` messages). The evaluator accumulates data in Python lists (`agents_list` and `robot_list`) during active recording sessions, which are controlled via ROS 2 services:

- `/hunav_start_recording`: Accepts `StartEvaluation` requests with experiment tags, robot goal poses, and run identifiers to begin data collection.

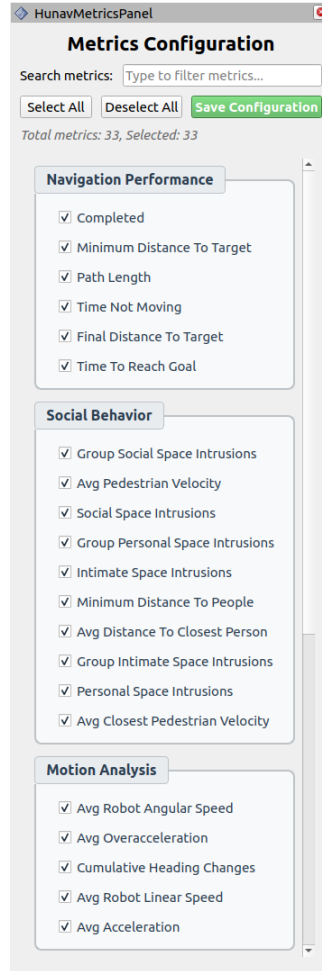


Figure 14: Metrics configuration panel providing interactive selection from a categorized catalogue of 33 evaluation metrics. The interface organizes metrics into six thematic categories (Navigation Performance, Social Behavior, Motion Analysis, Safety & Collisions, Social Forces, Danger & Surprise Metrics) with real-time search filtering, bulk selection controls, and persistent status display. Each metric includes detailed tooltips explaining theoretical foundations, units, and interpretation guidelines.

- `/hunav.stop_recording`: Triggers batch metric computation upon completion of the recording session.

Proxemic space intrusions are computed by evaluating robot-person distances against thresholds derived from Hall’s Proxemics theory: intimate space ($d < 0.45$ m), personal space ($0.45 \leq d < 1.2$ m), and social space ($1.2 \leq d <$

3.6 m). Group-aware metrics extend this framework by identifying spatial formations through computed interaction centers at configurable distances from agent positions, then evaluating intrusion events relative to group boundaries rather than individual positions. Social force metrics apply the Helbing-Molnár model to quantify repulsive forces between agents, between agents and obstacles, and their influence on robot trajectories. The social work function integrates these forces over the complete trajectory to assess cumulative social appropriateness.

Collision detection distinguishes between robot-on-person events (where the robot’s velocity vector caused contact) and person-on-robot events (where a pedestrian’s velocity vector caused contact) by analyzing relative angles and velocities at collision points. This enables differentiation between navigation failures and unpredictable human behaviors. Danger and surprise metrics evaluate psychological comfort by computing fear and panic costs from velocity-based threat assessments, visibility-based surprise from relative positioning within a 120° field of view using proxemic distance thresholds (default 0.45 m), and shock/reaction costs from temporal analysis of unexpected encounters with configurable recognition times (default 150 ms) and reaction times (default 600 ms). For surprise metrics requiring line-of-sight calculations, the evaluator retrieves an occupancy grid from the `/map_server/map` service (see Figure 15), which is converted to a numpy array for efficient spatial queries.

All metrics are computed with configurable sampling rates (default 1 Hz, or at message publication frequency if set to 0) and logged with nanosecond-precision timestamps for fine-grained temporal analysis.

6.3 Results generation and analysis

As shown in Figure 15, upon receiving a `/hunav_stop_recording` service call, the evaluator node triggers metric computation by iterating through all enabled metrics and invoking their corresponding computation functions from the `hunav_metrics` module. The system generates structured output in CSV format with a primary results file containing per-metric numerical values with associated metadata (experiment tag, run identifier, timestamp), enabling automated parsing and comparison across experimental runs. For time-series metrics that return multiple values (e.g., per-timestep distances), the system generates a secondary “steps” CSV file with complete temporal sequences indexed by timestamps.

The results file structure uses Pandas DataFrame serialization to CSV format, with experiment tags serving as row indices and metric names as column headers. For multi-behavior scenarios, the system generates per-behavior breakdowns by filtering agent messages according to their behavior type attribute, producing separate CSV files with `_beh_{id}` suffixes. This supports evaluation of how different human behavioral models (curious, scared, threatening, etc.) influence navigation metrics.

The benchmarking system supports batch evaluation through programmatic manipulation of the `StartEvaluation` service requests. Users may define sce-

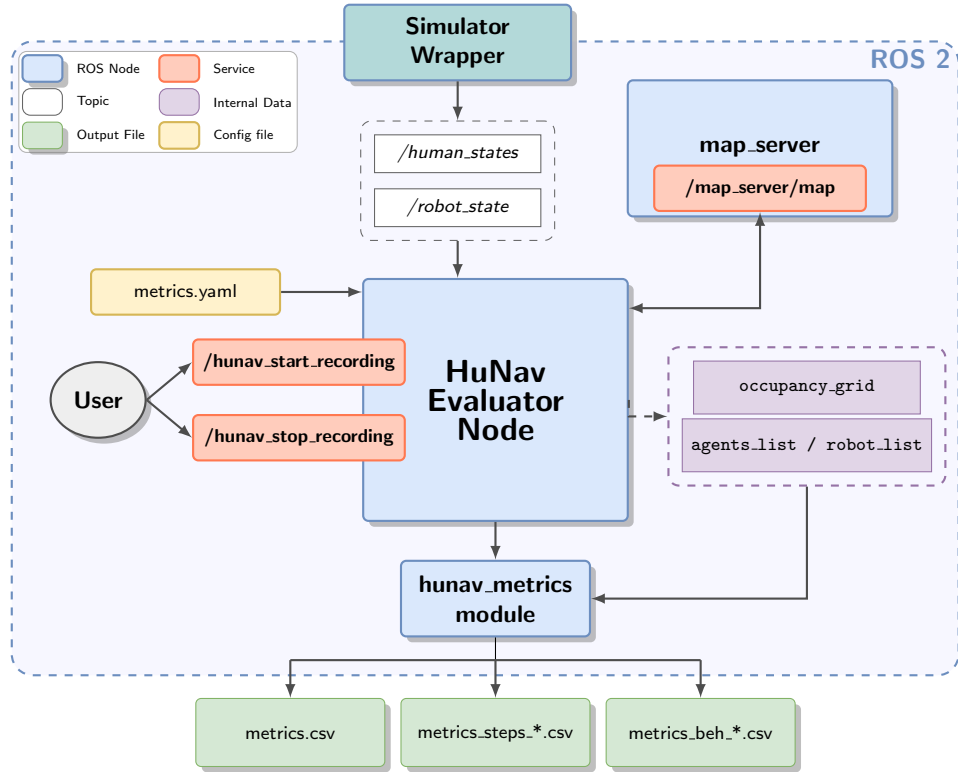


Figure 15: Architecture of the HuNav Evaluator. The system integrates real-time agent/robot states and static map data (via service) to compute social navigation metrics. Data is accumulated in internal structures, processed by the metrics module, and exported to structured CSV files.

nario parameter sweeps in external scripts, repeatedly invoking the start/stop recording services with different experiment tags and run identifiers. The CSV append mode ensures that multiple experimental runs accumulate in a single file for streamlined comparative analysis.

6.4 Adding new metrics

The metrics framework provides a modular architecture for user-defined extensions through a three-step process. First, metric definitions are added to the configuration file with boolean enable/disable states. Second, computation logic is implemented as a Python function that accepts the time-series states of all humans and the robot, and optionally an occupancy grid for map-dependent calculations. The function returns a list with the scalar metric value as the first element and an optional time-series as the second element. Third, the metrics panel automatically discovers and displays new metrics at initialization—requiring no GUI modification.

For example, a simple path-based metric computes the total distance traveled by the robot:

```
1 def robot_path_length(agents: List[Agents], robot: List[
  Agent]) -> List[float]:
2     """Computes the total path length traveled by the robot.
      """
3     path_length = 0.0
4     for i in range(len(robot) - 1):
5         path_length += euclidean_distance(robot[i+1].
          position, robot[i].position)
6     return [path_length]
```

More sophisticated metrics leverage complete state information from both parameters. The following implements the danger fear cost from Singamaneni et al. [6], quantifying human discomfort based on relative positions and velocities:

```
1 def danger_fear_cost(agents: List[Agents], robot: List[Agent
  ]) -> List[int]:
2     """Computes accumulated fear cost across all timesteps
      and agents."""
3     fc = 0.0
4     fc_list = np.zeros(len(agents[0].agents)) # Per-agent
      accumulator
5     for agts, rb in zip(agents, robot):
6         robot_pos = np.array([rb.position.position.x, rb.
          position.position.y])
7         robot_vel = np.array([rb.velocity.linear.x, rb.
          velocity.linear.y])
8         for idx, agent in enumerate(agts.agents):
9             human_pos = np.array([agent.position.position.x,
              agent.position.position.y])
10             human_vel = np.array([agent.velocity.linear.x,
```



```

12         agent.velocity.linear.y])
13     f = cost_fear(robot_pos, robot_vel, human_pos,
14                 human_vel,
15                 rh=agent.radius, rr=rb.radius)
16     fc += f
17     fc_list[idx] += f
18     return [fc, fc_list.tolist()] # Total cost + per-agent
19     breakdown

```

Complex metrics requiring additional dependencies may extend the evaluator node by adding topic subscriptions (e.g., for map data as demonstrated by the surprise metrics), maintaining internal state variables, and implementing custom logic. New entries are automatically recognized without modifying core node logic.

Documentation for each metric should include a title, description of what is measured, theoretical or empirical foundation (with citation if applicable), physical units, and interpretation guidelines (e.g., “lower values indicate better performance”). This ensures comprehensive self-documentation for both standard and user-contributed metrics, promoting reproducibility across research groups.

The integrated benchmarking system transforms social navigation evaluation from a manual, error-prone process to a standardized, automated workflow while preserving extensibility for domain-specific needs. Complete metric definitions and implementation examples are maintained in the `hunav_evaluator`⁶ package documentation.

References

- [1] Helbing and Molnár, “Social force model for pedestrian dynamics.” *Physical review. E, Statistical physics, plasmas, fluids, and related interdisciplinary topics*, vol. 51 5, pp. 4282–4286, 1995.
- [2] N. Perez-Higueras, F. Caballero, and L. Merino, “Teaching Robot Navigation Behaviors to Optimal RRT Planners,” *International Journal of Social Robotics*, vol. 10, no. 2, pp. 235–249, 2018.
- [3] N. Tsoi, A. Xiang, P. Yu, S. S. Sohn, G. Schwartz, S. Ramesh, M. Hussein, A. W. Gupta, M. Kapadia, and M. Vázquez, “Sean 2.0: Formalizing and generating social situations for robot navigation,” *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 11 047–11 054, 2022.
- [4] A. Biswas, A. Wang, G. Silvera, A. Steinfeld, and H. Admoni, “Socnavbench: A grounded simulation testing framework for evaluating social navigation,” *ACM Transactions on Human-Robot Interaction*, jul 2022. [Online]. Available: <https://doi.org/10.1145/3476413>

⁶https://github.com/robotics-upo/hunav_sim/tree/v2.0/hunav_evaluator

- [5] K. Katyal, Y. Gao, J. Markowitz, S. Pohland, C. Rivera, I.-J. Wang, and C.-M. Huang, “Learning a group-aware policy for robot navigation,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 11 328–11 335.
- [6] P. T. Singamaneni, A. Favier, and R. Alami, “Towards benchmarking human-aware social robot navigation: A new perspective and metrics,” in *2023 32nd IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*, 2023, pp. 914–921.

Table 2: Available metrics in *HuNavSim*

Metric	Source	Description	Units
time to reach goal	Pérez et al. [2]	Time spent by the robot to reach the goal.	s
path length	Pérez et al. [2]	Total length of the robot trajectory.	m
cumulative heading changes	Pérez et al. [2]	Sum of absolute heading angle differences between successive waypoints.	rad
avg dist to closest person	Pérez et al. [2]	Average distance to the closest person at each time step.	m
min dist to people	Pérez et al. [2]	Minimum distance to any person along the trajectory.	m
intimate space intrusions	Pérez et al. [2]	Percentage of time the robot spent within a person's intimate space (proxemics).	%
personal space intrusions	Pérez et al. [2]	Percentage of time the robot spent within a person's personal space.	%
social space intrusions	Pérez et al. [2]	Percentage of time the robot spent within a person's social or public space.	%
group intimate space intrusions	Pérez et al. [2]	Percentage of time the robot spent within a group intimate space.	%
group personal space intrusions	Pérez et al. [2]	Percentage of time the robot spent within a group personal space.	%
group social space intrusions	Pérez et al. [2]	Percentage of time the robot spent within a group social or public space.	%
completed	SEAN2.0 [3]	Whether the robot successfully reached the goal.	–
min dist to target	SEAN2.0 [3]	Minimum distance between the robot and the target position.	m
final dist to target	SEAN2.0 [3]	Distance between the final robot position and the target.	m
robot on person collisions	SEAN2.0 [3]	Number of collisions between the robot and people.	–
person on robot collisions	SEAN2.0 [3]	Number of collisions between people and the robot.	–
time not moving	SEAN2.0 [3]	Total time during which the robot was stationary.	s
avg robot linear speed	SocNavBench [4]	Average linear speed of the robot along the trajectory.	m/s
avg robot angular speed	SocNavBench [4]	Average absolute angular speed of the robot.	rad/s
avg robot acceleration	SocNavBench [4]	Average acceleration of the robot.	m/s^2
avg robot jerk	SocNavBench [4]	Average jerk (time derivative of acceleration).	m/s^3
avg pedestrians velocity	Katyal et al. [5]	Average linear velocity of pedestrians along the trajectory.	m/s
avg closest pedestrian velocity	Katyal et al. [5]	Average linear velocity of the closest pedestrian.	m/s
social force on agents	SFM-based [1]	Sum of social forces exerted by the robot on human agents along the trajectory.	m/s^2
social force on robot	SFM-based [1]	Sum of social forces exerted by agents on the robot.	m/s^2
obstacle force on agents	SFM-based [1]	Sum of obstacle forces acting on human agents.	m/s^2
obstacle force on robot	SFM-based [1]	Sum of obstacle forces acting on the robot.	m/s^2
social work	SFM-based [1]	Sum of social and obstacle forces acting on the robot and agents.	m/s^2
danger fear cost	Singamaneni et al. [6]	Cumulative fear cost based on relative velocity and effective distance between robot and humans.	–
danger panic cost	Singamaneni et al. [6]	Cumulative panic cost based on collision time and effective distance.	–
surprise visibility cost	Singamaneni et al. [6]	Cumulative cost for robot appearing within human's field of view at close proximity. Requires occupancy map.	–
surprise shock cost	Singamaneni et al. [6]	Cumulative shock cost based on recognition and reaction time when robot becomes visible. Requires occupancy map.	–
surprise react cost	Singamaneni et al. [6]	Cumulative reaction cost based on time to react after robot becomes visible. Requires occupancy map.	–